

Splitting incubation_app.py

Why it matters, and how to do it safely in a coordinated window · Bee Incubation Manager · Tyler & Darren · 19 July 2026

1. Where we are today

One file holds nearly the whole desktop application: the dashboard, incubators, samples, trays, timeline, inspections, settings and incubator-detail views, plus alerts, Govee polling, the QR/phone server wiring, git auto-sync, backups, email reports and calendar sync.

incubation_app.py	5,425 lines	2.2x the next-largest file
qr_server.py	2,474 lines	next largest
Commits touching incubation_app.py	58 in 30 days	~2 per day, from two developers

2. Why we should do this

- **It is our number-one merge-conflict hotspot.** Two developers both edit this single 5,400-line file almost every day. The more we each touch it, the more often our edits land on the same lines.
- **With auto-sync, a conflict is expensive, not just annoying.** The app syncs code automatically using `git pull --ff-only`. The moment the two histories diverge, that pull refuses to run and **auto-sync stops entirely** on that machine until a human resolves it by hand. Code silently stops flowing between computers - including to the machine that serves the phone app.
- **It makes far more of the app testable.** We now have a pytest suite (28 tests), but it can only cover pure logic. Logic buried inside a 5,400-line GUI class cannot be tested at all. Splitting it out lets us move real logic behind testable seams and grow that safety net - which matters because auto-sync pushes changes to every machine.
- **Smaller blast radius.** When Settings and the Dashboard live in different files, a change to one cannot accidentally break the other, and a bad change can be reverted on its own.
- **Better tooling and AI assistance.** Editors, code review and Claude Code all work markedly better on focused files: less context to load, fewer accidental edits elsewhere in the file, and much faster, more reliable changes.
- **Easier to navigate and onboard.** Finding the Settings screen should mean opening `views/settings_view.py`, not scrolling through 5,400 lines.
- **It only gets harder.** This file grew substantially in just the last month. The cheapest time to split it is now, while we still understand every part of it.

3. Why it needs a coordinated window

This is the one refactor that should **not** be dropped in unannounced. A split touches the whole file, so its diff overlaps almost any work in flight. Three specific hazards:

- Any uncommitted work the other person has in that file will conflict, and they will have to resolve it by hand.
- Divergent history pauses `--ff-only` auto-sync on the other machine, so code stops syncing until it is fixed.
- Our tests cover logic but **not the GUI**, so a wiring mistake would not be caught automatically - it has to be caught by clicking through the app.

All three disappear if we do it in a short window when only one person is editing.

4. The method: mixin extraction (a move, not a rewrite)

Move cohesive groups of methods out of the App class into modules under views/, as mixin classes that App inherits. Every self. reference keeps working exactly as before - this is relocation, not redesign, which is the lowest-risk way to break up a large GUI class.

```
# views/settings_view.py
class SettingsViewMixin:
    def _build_settings_view(self): ...
    def _refresh_settings(self): ...
    def _save_settings(self): ...

# incubation_app.py
class App(ctk.CTk, DashboardViewMixin, SettingsViewMixin, TraysViewMixin, ...):
    ...
```

Suggested order - most self-contained first:

Settings → Samples → Trays → Timeline → Inspections → Dashboard + Incubators → Incubator detail → background services (git sync, backups, alerts, Govee). Leave the detail view and background services for last - they have the most cross-references.

5. The coordinated window, step by step

Before - both of us (about 10 minutes)

- 1 Agree a window of roughly 60-90 minutes and pick **one driver**. Only the driver edits code during the window.
- 2 Both: finish and **push** everything. git status must be clean on both machines.
- 3 Both: **turn auto-sync off** - set auto_git_sync to 0 in Settings, or simply close the app. This stops it committing or pushing mid-refactor.
- 4 Both: run git log -1 --oneline and confirm you are on the **same commit**. Write that SHA down - it is the known-good rollback point.

During - driver only

- 1 Work on a branch: git checkout -b refactor/split-views.
- 2 Extract **one view at a time**. Never batch several together.
- 3 After each extraction, verify: python -m py_compile incubation_app.py, then python -m pytest tests/ -q, then **launch the app and click through every view** (the tests do not cover the GUI).
- 4 Commit each extraction on its own, with a clear message such as "Extract Settings view into views/settings_view.py".
- 5 Only push once every view has been opened and checked.

After - the other person

- 1 git pull - it fast-forwards cleanly because your tree was clean.
- 2 Restart the app and click through every view.
- 3 Both: set auto_git_sync back to 1 and confirm sync resumes.
- 4 If the phone site is hosted on either machine, restart the app there too so Flask reloads the new code.

6. If something goes wrong

- Each extraction is its own commit, so git revert <sha> undoes just that one view without touching the rest.

- Worst case, reset to the pre-refactor SHA recorded in step 4.
- Because behaviour is unchanged - the code only moves - any breakage shows up immediately when clicking through the views, not days later.

7. Definition of done

- The app launches on both machines and every view opens and works.
- `pytest` is green.
- Both machines pulled cleanly and auto-sync is back on and syncing.
- `incubation_app.py` is reduced to the app shell and wiring (target: under about 800 lines), with each view in its own module.

Realistic effort: one focused session (1-2 hours) covers the first four or five views; a second short session finishes the rest. It does not have to be done in one sitting - each extraction is independently safe once pushed.